YAML

```yaml
name: webdev
description: "Build and deliver an initialized Manus website, mobile app or gam
```

Webdev

Workflow

Local Web/Mobile requests follow Work Locally for rediscovery and managed-versus-plain-local choice before entering this Addon workflow.
Respect that choice; local Game keeps its existing managed workflow.

Use the project and setup state returned by init/attach. Continue an attached project's current
request and accepted decisions without repeating initialization or planning. Project-file locators
are relative to the returned project directory; documentation links are relative to this Skill’ s directory.
Game follows its delivered Game workflow. Web and Mobile share the common workflow and tool
rules below. Each selected template README adds its specific facts and
overrides; Mobile owns the fixed starter, device/backend, development
and app-build workflow. Web-specific setup, routing and publication rules do not apply to Mobile.

1. Plan

If a Blueprint is confirmed, it is the approved plan: skip writing `plan.md` and plan approval, but still read required guides and apply Design directly before ToDo.

For infrastructure and services needed by the task, default to Manus-provided capabilities unless the user explicitly requests an alternative.

Before planning, select required guides from the guide facts and user request.
Read known paths in one parallel tool-call batch; reuse complete guidance in context.

When you write an implementation plan, keep implementation and design decisions in one `plan.md`.
Explain how to implement the requested product and
how its parts fit together: the implementation approach, dependencies, material

constraints,
and frontend/backend serving. Preserve concrete requirements and distinguish required behavior
from optional ideas.

Include a project structure section in the plan, describing the main folders, modules, and their responsibilities. Ground these decisions in the project's actual capabilities, template guidance and existing files. Small changes reuse the existing plan and design; do not introduce a new planning pass just to satisfy this Skill.

Revise changed planning assumptions before implementation. Keep the plan focused on implementation
and necessary design decisions. Do not add a separate validation plan or acceptance checklist,
or turn product requirements into step-by-step test tasks.

Design

Reuse the Blueprint, user's style/reference, or existing project style; otherwise list three distinct styles (at most one short sentence each) and their probabilities (< 0.10, sampled at random from the tails of the distribution) in `plan.md`, choose one, and expand only that style below in the user's language:

- **Design Movement**: Reference a specific aesthetic.
- **Core Principles**: 3–4 defining characteristics that guide all design decisions.
- **Color Philosophy**: The palette's reasoning and emotional intent, not just color values.
- **Layout Paradigm**: A distinctive structural approach; avoid grid-based centralized layouts
  as much as possible.
- **Signature Elements**: 2–3 distinctive visual motifs used throughout.
- **Interaction Philosophy**: How interactions reflect the design ethos.
- **Animation**: Detailed animation guidelines that reflect the design ethos.
- **Typography System**: Specific font pairings and hierarchy rules that embody the style.
- **Brand Essence**: One-line positioning (what it is, who it is for, why it is different)
  plus 3 personality adjectives.
- **Brand Voice**: How headlines, CTAs and microcopy sound, with 2 example lines. Avoid generic
  filler such as "Welcome to our website" or "Get started today".

- **Wordmark & Logo**: A distinctive logotype/mark concept, not the brand name in a default font.
- **Signature Brand Color**: One ownable color that is unmistakably this brand's.

Keep this design description in `plan.md`.

2. ToDo

After plan approval, create or update project `TODO.md` before configuration or implementation. Replace
preliminary phase headings with outcome items; retain existing progress
only where the same outcome has already been delivered. Carry the
actual acceptance clauses from the request and approved plan into outcome items; copy clear
source clauses rather than summarizing them into one-line feature headings. Preserve concrete values, messages, field rules, actors, transitions
and exceptions, using the original request where the plan omitted or weakened a requirement.
Exclude secret API keys and implementation rationale from the copied clauses.

Keep each outcome heading and its complete criteria together in the item;
summaries or links to the plan do not substitute for these clauses. For unstructured requests,
identify independently deliverable outcomes before grouping. Merge duplicate clauses, keep
distinct feature groups separate, and avoid both a single whole-application item and an item per
field. Do not force a tiny item count. Before creating the list, compare it with the source: every required condition must remain explicit, with duplicates consolidated rather than generalized.
Preserve outcome items and full criteria on status updates. Acceptance clauses describe product
behavior, not additional tests. Reuse code and existing evidence; do not add a separate testing
task for every product requirement. Mark an item complete only with evidence of its outcome;
keep unresolved outcomes open. For complex tasks, include the validation agent described below.

3. Implement

Prefer a well-defined project structure over writing a very large amount of code in a single file.
Write readable source code in the project's existing style, with appropriate line breaks and indentation for the language.

Login in embedded Preview

For any website with cookie-based login, **set the application's session cookie to `SameSite=None; Secure` for public HTTPS Preview**. Manus embeds Preview in a cross-site iframe; `SameSite=Lax` or `Strict` prevents that cookie from accompanying iframe requests.
Do not select these attributes from `NODE_ENV` or the development server's internal `req.secure`: public HTTPS can reach that server over HTTP. Keep plain-HTTP local handling separate.
Browser third-party-cookie policies can still block cross-site cookies.

Project-logo metadata

Platform branding is separate from site favicons and native icons. Before checkpointing, set a quoted `logoUrl` literal in project-root `app.config.ts`, preserving existing fields; if absent, create `export default { logoUrl: "<actual HTTPS URL>" }`.
Use a durable HTTPS asset URL or `manus-upload-file <logo-file>` without `--webdev`.
The hook reads the accepted commit and does not evaluate expressions; no import is needed.
Limits: URL 2056 characters, no credentials or outer whitespace; file 1 MiB.
Relative paths and temporary signed URLs are invalid; failed sync keeps the old logo.

Image assets

- Reuse suitable images supplied by the user or already in the project. If the user supplies a style reference, use it to guide the design; do not search again merely because a reference was supplied.
- When an image must depict a specific real person, product or place, use the supplied photo or search for an accurate one; do not generate a lookalike.
- For custom hero images, banners or brand illustrations, use the Agent's `generate_image` tool. For a new site, decide their page placements and submit independent image requests together before application coding. Describe the subject, composition, colors and style in each prompt. Generate only images with a planned use; honor an exact count requested by the user. Do not substitute generic stock images for these custom visuals.
- For internal tools and dashboards, skip extra decorative illustrations unless requested. This does not exclude images required by the product, such as product photos or user

avatars.

- Use returned image URLs unchanged and continue independent work while generation runs. Do not poll or resubmit pending requests, or claim completion before the completion result. Do not fill unrelated sections with the same image; remove placeholder imagery before delivery.

File uploads

Default to `manus-upload-file --webdev <file>`. Upload website images and attachments to project storage, and use the returned `/manus-storage/<key>` path in the website.

Use `manus-upload-file <file>` without `--webdev` only when the consumer requires an absolute public URL, such as platform `logoUrl` or Shopify media ingestion. It returns a public CDN URL
independent of the Webdev project lifecycle; the project-relative path is not suitable there. Reuse an existing suitable URL instead of uploading again.

Development service

Set `cwd` to the absolute `project_dir` for project commands; use `service=true` for resident development servers. Start the server once the interface has meaningful content, and verify readiness with an HTTP request.

Cloud uses the project's configured runtime port on `0.0.0.0`; Local uses the Session's declared
port on `127.0.0.1`. Do not automatically switch ports. Each Cloud Session has its own Sandbox;
after attach, restore missing dependencies and development services. Reuse existing healthy services.
A migrated website may have no configured Preview port yet. When Preview is needed, inspect the
existing project's development command and service, then declare its port through `webdev.config PUT config/runtime` before using Preview; do not reinitialize the project.

Routine diagnostics

**Before writing the first batch of application code, ensure language diagnostics are configured
for the project's languages.** Inspect `webdev.config` with `GET runtime/post-edit`, reuse matching
registrations, and complete missing setup using the applicable flow below. Do not wait for a build or typecheck failure to enable LSP.
Use the exact tool path `runtime/post-edit`, without a `config/` prefix. In Sandbox, confirm

`registered: true` and the required `languages` in the successful response before proceeding. Absence of an edit reminder does not prove that checking completed or that the code is clean.

Host-managed diagnostics

When `GET runtime/post-edit` returns registered `languages`, the host owns language-server processes and checks registered project files after edits. Web and mobile starter projects register TypeScript diagnostics during initialization. Do not launch `manus-webdev-lsp` or a second native server in Sandbox.

Use `webdev.config` with `GET runtime/post-edit` to inspect the registered languages. To select project languages, use `PUT runtime/post-edit` with `{"languages":["typescript"]}` (supported: typescript, javascript, python, go, rust, json, yaml, html, css). `DELETE runtime/post-edit` removes diagnostics for this project; it does not remove its workspace or preview. Sandbox returns code diagnostics with the edit result. A pending or unavailable check is not a clean result.

The remaining instructions apply to Device and Operator runtimes where `GET runtime/post-edit` returns `servers`; these use the existing agent-managed server and MCP client.

Agent-managed diagnostics

Manus provides the file-edit hook and a built-in LSP client inside the Webdev MCP server. You do not install, launch, version, or configure a separate LSP client or HTTP adapter.

Manage the native server/process/settings. Presets provide prepared starting points; other standard LSP servers use the custom-server guide.

Start a preset

Use the `launcher` argv returned by `GET runtime/post-edit`: its runtime executable runs the supplied script. Append the subcommand and arguments shown below; `manus-webdev-lsp` in examples stands for this complete launcher, not the raw `cli_path`. Quote paths for your shell; PowerShell needs `&` before a quoted executable path. This works without a global PATH command or executable permission on the script. If no launcher is available, use the custom-server contract.

Start the language server as a resident process using `exec` with `service: true` and an explicit project working directory:

Plain Text

```
manus-webdev-lsp start --preset typescript --project /home/ubuntu/app
```

The preset starts the server, prepares editable settings and registers/initializes the client. Its receipt reports the process, connection, file mapping, settings file and actual status. `ready` means protocol initialization succeeded, not that every edit has been checked.

Use `manus-webdev-lsp list` to discover presets and `manus-webdev-lsp describe --preset NAME` for the native settings descriptions and examples. Use `--settings /absolute/path.json` to supply your own effective protocol settings. An existing settings file must not be overwritten with preset defaults on restart.

Do not run a second HTTP `serve` process or manually repeat a successful preset registration. The resident service remains running; do not wait for it to finish as a finite task.

Resolve actionable diagnostics; if reliable LSP checking is unavailable, use the language's existing check-only command. An actionable diagnostic on a real source file and its removal after correction establish that a newly selected setup is checking; process readiness alone does not.
Custom server registration, native settings/reload, recovery and published logs are in Diagnostics; select that page in the planning batch when the task already needs those operations.

Website route manifest

Before the first Web dev-server start, serve a static JSON document at `GET /manus-routes.json` on the website origin, e.g. `public/manus-routes.json`. Derive the complete page route set from current source, including nested/lazy routes and dynamic patterns such as `/todo/:id`; exclude APIs, assets, system endpoints and 404-only routes. Keep it synchronized with route changes.

```JSON
{"routes":[{"path":"/","title":"Home"},{"path":"/todo/:id","title":"Todo"}]}
```

Only top-level `routes` and entry fields `path` /optional `title` are accepted. Paths must start with `/`; queries/hashes are allowed, absolute URLs, `//`, credentials, raw whitespace and control characters are not. Limits: 1 MiB, 1000 routes, 2048 characters per path (also after URL encoding), 256 per title. Never truncate the set or invent entity IDs. Request the file after startup/route changes: require HTTP 200 and this JSON, not SPA fallback HTML, and compare with source. These page declarations are separate from published request routing below.

4. Validate before delivery

The project's managed database is shared by development, testing, and production. Writes and deletions during testing also affect production data.

Prefer code inspection, automatic diagnostics and existing checks; avoid browser testing and
screenshots unless explicitly requested or needed to investigate a concrete observed defect.
For complex tasks, use one read-only `agent` with `sandbox="shared"` to check the implementation
against requirements and trace how application parts connect.
Provide the project directory, plan/ToDo, relevant platform facts and existing evidence;
the child does not inherit the parent conversation.
Fix confirmed issues and recheck affected paths; do not repeat full acceptance passes.

Webdev tool parallelism

**DEFAULT TO PARALLEL TOOL CALLS. In EVERY response, batch ready read, write, edit, exec and
MCP calls within the host limit; mix tools and repeat the SAME tool.** Emit intended order; the
host preserves required ordering. A write alone needs no new model turn.

Wait only for a prior result needed to choose an action (unread guide, unknown config, returned
ID/status), a standalone tool (init/attach or exclusive), pending user/Host input, or unconfirmed
binding, runtime sync or checkpoint. Group up to 12 known secret keys. Do not queue competing
cards or blocked writes; do not race background/session changes to config, ports, files or Git.
Otherwise, batch the calls. Continue independent work during Publish without polling or duplication.

Configuration

`webdev.config` operates the bound project; application code uses runtime APIs. The tool supplies
project identity, authentication and Skill version. Reuse init/attach facts; `GET config` before writes, recovery or when fields are missing or concurrent changes matter.

Prefer `PUT config/{domain}` with the complete domain fragment, e.g. `{"runtime":{"port":3000}}`; it replaces only that domain, so preserve its unchanged fields. Secret declarations change only

their named key; unrelated scoped updates do not share a project revision. Whole `PUT config`
replaces the document: include `version:1`, `runtime` and all intended fields. Read Configuration details for whole-document imports or interactive
recovery. Do not combine Git transitions or
new secret candidates with unrelated changes. Integration enablement uses its dedicated route.

A successful `{ok:true,revision:N}` means stored; `runtime_sync: applied/noop` means locally loaded. On `failed`, correct the cause and retry `GET config` once before relying on new values; existing processes may need restart. For `revision_conflict`, inspect affected whole-document, dependency or pending state before retry; unrelated scoped writes do not cause it. Read `reason_code`/field errors rather than blindly repeating writes.

Use `webdev.request_secrets` for protected user input and follow its card/wait behavior. With pending input, `blocks_config_writes:false` allows unrelated scoped writes; `true` is a legacy all-write lock. Conflicting keys, dependencies and whole-document replacement wait or use returned
`unblock`; do not retry. `action_url` alone does not mean stop: follow Host pause/notifications. Stripe claim is non-blocking. Completion applies the change; do not repeat a completed replacement.

Web `features` accepts only `server` and `database`. Both start off; later omitted keys preserve effective values. `server:true` does not request a database. `database:true` also enables server when omitted; explicitly pairing it with `server:false` is invalid. Enablement is one-way.
For requested Manus login select server during init, then read Authentication before implementation.
Game/Mobile retain their distinct feature policies.

`runtime` requires `port` (1024–65535); preserve `post_edit`. Optional `endpoints` contains 1–4 distinct `{name,port}` entries; names match `^[a-z](?:[a-z0-9-]{0,30}[a-z0-9])?$` and exclude `primary` / `preview`; ports differ from the primary port. Reserved: 5900, 5901, 8328, 8330, 8340,
8350, 9222, 9330, 19780, 50031. In Studio, only Session-local `runtime.port` is writable here; keep GET's effective port unchanged in whole-document writes. Preview extras and recovery:
Runtime. Configuration never starts the application for you.

Deployment

Prepare the matching declaration while implementing; reuse valid init defaults.

Published serving	Required config

Static output	`build`
Container	`features.server:true` and `deploy`
Static frontend + container backend	`build`, `features.server:true`, `deploy`, and path-splitting `routes`

`build` is `{"build":{"command":"<self-contained install/build>","outputDirectory":"<relative directory>"}}`. The clean checkpoint build does not implicitly install dependencies. Include the pinned toolchain/
lockfile installation when needed; output must contain `index.html`. No-build HTML uses `command:"true"`
and its committed output subdirectory. Clone, static build and upload share a five-minute budget.

`deploy` names `dockerfilePath` (default root `Dockerfile`) and required `healthPath` (unauthenticated
2xx–3xx). Store server enablement before a separate deploy write, or include both in one atomic
whole-document update. Optional top-level `buildCache` controls container image caching and
requires deploy; it defaults to enabled.

Command: nonempty, at most 1000 characters, no NUL or inline credential literals. Build output/
Dockerfile paths: repository-relative, 1–256 characters from `A-Za-z0-9._/-`, no absolute path, `.` / `..` or empty segments, or trailing slash; symlinks stay within the repository. Health path: starts `/`, 1–256 characters from `A-Za-z0-9._/-`, no `..` or NUL.

Build and production runtime

Build from the recorded checkpoint, not the Sandbox working directory. The platform replaces
`.dockerignore`, excluding `node_modules`, `.git`, `.env` and `.env.*` while keeping `.env.example`. Sandbox-installed dependencies are not image inputs.

The Dockerfile must install dependencies, compile the application and define its production entrypoint. Container static roots must exist in the final image. Building frontend files in the image does not publish separate static output; declare `build` for that output.
Static builds run the declared command at the repository root with the publication environment.

Do not depend on private credentials or `DATABASE_URL` being available during image compilation;
read them at runtime. Only browser-safe values may be included in frontend output.

Keep Dockerfile `EXPOSE` consistent with the application's listening port, and honor `PORT` when supplied; use 3000 by default. The published listener is separate from Preview's `runtime.port`; do not declare `deploy.port`. Container publication uses `healthPath` for HTTP readiness; serve an unauthenticated success response there.

Container memory and writable files are temporary. Store durable data in the database or object
storage. New and old instances may overlap during rollout; publication is not an immediate shutdown of the old version.

Published routing

Published `routes` replace all 1–32 ordered `{path,target:"static"|"server"}` rules.
Put APIs, health/webhooks and SSR before static catch-all; static needs build, server needs capability.
Use `webdev.config`: GET `config`, merge rules, then PUT.
Pure-static cache-only rules match files (`/styles.css`) or asset-only prefixes (`/css/*`), never `/*`:
routed `/dir` skips `/dir/index.html`; unmatched paths keep defaults.
Hybrid example, no existing table:

```
JSON

{"method":"PUT","path":"config/routes","body":{"routes":[{"path":"/api/*","targ
```

Paths start `/`, use `A-Z a-z 0-9 . _ ~ -` per segment, max 256 characters, optional final `/*`.
Reject bare `/`, dot-only/empty segments, trailing slash, mid-path wildcards, queries/fragments,
whitespace, duplicates and shadowed rules.
`{}` removes the table; empty arrays are invalid. Static-only SPA fallback must not mask missing assets/APIs. Reserved paths/subpaths: `/favicon.ico`, `/manus-oauth/`, `/api/scheduled/`, `/manus-storage/`, `/__manus/`, `/__system__/`, `/sitemap.xml`, `/robots.txt`; platform handling takes priority even under broader wildcards.
Routes apply after publish, not in Preview; no User-Agent/header matching.

Before publish, use `no-cache` for mutable fixed URLs; `immutable` requires all matched URLs to change with content (hash/version), even under `/assets/`. Static cache accepts these keywords
or 0–31536000 seconds; omission means 90 days for non-HTML.

Server cache: `no-cache` or 0–600 seconds (0 disables); omit to keep behavior.
Shared server HTML cache requires public production GET/200 HTML outside `/api/`; Cookie/Authorization, private/no-store/no-cache, Set-Cookie or unsupported Vary block storage.
App may omit Cache-Control or send `public,max-age=0`; browsers get no-store.
Cache only visitor-independent content; sensitive responses use private/no-store.
Runtime/build failures, CSP/HTML injection and cache checks:
build, response.

Checkpoints and publication

A **checkpoint** is a saved website version recorded by Manus, identified by a Git commit SHA and shown in Version history. It records a commit accepted on `main` in the project's currently
selected repository (the canonical repository). A local commit or a push to another branch is not a checkpoint. Saving a checkpoint and publishing it are separate operations.

Reuse the known `publishing.auto_publish` state; read `GET config` before saving only when the state is unknown or the user asks not to publish.

- **Cloud:** For each deliverable change, commit the intended files in `project_dir`. Fetch `origin/main`, integrate any remote changes while preserving both sides, run affected checks,
  and push the intended commit to `origin/main`. The platform automatically records the checkpoint
  after the Git command; no separate save tool is needed. Confirm the intended SHA in the returned
  version card or Version history, not just Git exit zero.
- **Local:** Use `webdev.manus_git` under the already selected Work Locally workflow. Fetch and
  integrate its returned `<remote_refs>/main`, then push through the temporary `manus` remote;
  preserve the user's separate Git. Confirm the result's `checkpoint` is `recorded` or `already_recorded`. A push to the user's separate repository is not a Manus checkpoint.

Never force-push or rewrite the project's main history. After an interrupted operation, inspect
its actual result before retrying; do not repeat the whole Git command just to retry recording.

`publishing.auto_publish:true` is the user's continuing authorization for checkpoint-triggered publication; never enable it yourself. If the user says not to publish while enabled, ask them to turn off the Dashboard switch before a triggering push. Auto-publish completion appears

in
the Dashboard; do not submit another publish for the same work. Explicit publication, rollback
and Git recovery use Publishing, rollback and Git recovery.
MCP Publish is a background job returning its final result automatically; continue independent
work without polling.

Delivery

For an ordinary website handoff, keep the final reply within 100 words unless detail is requested.
State the result and blockers; include the published URL only when the delivered version's successful publication is confirmed. Never include Preview URLs in chat messages; direct the
user to Preview instead. Preview/checkpoint success is not publication success.
Provide file attachments only when requested or when the file itself is a requested deliverable.
Refer to checkpoints by their returned SHA/card/Version history; do not invent Resource URIs or closing tools.

Planning read set

These facts expose all explicit/implicit dependencies for the whole task, including later implementation, verification and delivery. Cross-links locate owners.

Templates, environment and toolchains

Guide	Facts and dependencies
web-db-user README	Starter commands, code and resource-dependent setup. Supplied helpers use Authentication, Database, Storage and Service API capabilities below; their presence does not enable those services.
Expo README	Fixed Mobile stack, two listeners, device credentials, icons/builds. Its app capabilities use Authentication, Database, Storage and Service API; Preview uses Runtime, local setup choice uses Work Locally.

Dependency setup	New Node toolchains/dependencies: pnpm pinning and lifecycle permissions before installation; blocked-script recovery.
Work Locally	Pre-init directory/permission choice, identity, separate Managed Git, environment loading and Preview. Local platform calls depend on that environment. Repository transfer uses Git canonical; Game has its separate workflow.

Capabilities

Guide	Facts and dependencies
Service API	Shared URLs, credentials, HTTP/RPC and errors for LLM, Images, Speech, Maps, API hub, Storage, Owner notifications and schedule-management RPCs. Owns platform signing names; independent keys use Secrets, build availability uses the main deployment contract. OAuth/Database/Stripe have separate transports.
Authentication	Any login, including self-managed email/password: application sessions, roles and cross-site Preview cookies. Manus OAuth is the default only when no provider is specified. Requires Server; user/role records use Database, independent credentials use Secrets, Preview proxy uses Runtime.
Database	Managed MySQL, migrations and shared development/published data; requires Server. No application tables or ownership model are supplied. External DSNs use Secrets; unusual startup preparation uses Build contracts.
Storage	Service API transfers and durable `/manus-storage/` paths. Stable Cloud Preview and published routing are platform-owned; direct dev-port reads bypass that route. Database owns indexes, Authentication access; Runtime Images need transfer, Agent images

	already exist, and LLM inputs may need signed downloads.
LLM	Built-in Manus AI is the default for requested AI text/chat generation. It uses project-provided credentials; no separate model-provider account or key is required. Read with Service API for chat completions, models, structured output and streaming. File/image inputs must be fetchable; Storage owns private-asset access. Responses do not persist conversations or implement accounts.
Images	Service API generation/editing and model catalog. Returns URL/base64, not a durable asset; persistence belongs to Storage. Agent artwork has a different managed-asset result.
Speech	Service API transcription returns text/timestamps. Recording retention belongs to Storage; application transcript records belong to Database.
Maps	Google browser/Web Service proxy with Service API credentials and Google-style key authentication. The main deployment contract owns build-time public values; results are not persisted application data.
API hub	Service API calls to managed external data APIs; registered API search supplies discovery. Separate provider keys use Secrets.
Owner notifications	Service API messages to the project owner only; no customer-email channel or business-event store. Separate provider credentials use Secrets; durable records use Database.
Scheduled work	Published callbacks, retries and task identity. Callback authentication is self-contained. Management RPCs use Service API; task associations use Database; AI work uses LLM. Publication uses Git/checkpoints; lifecycle/routes use the main deployment contract.
Payments	Stripe provisioning, checkout, webhooks and live handoff; requires Server. App identity/entitlements use Authentication/Database. User-input keys

	use Secrets; advanced declarations/pending state use Configuration. Mutually exclusive with Shopify.
Shopify	Storefront stack, store choice, Admin and claim flow; relevant before init/stack selection. Excludes Stripe. Product-image assets use Storage; independent credentials use Secrets.
Secrets	User-input keys, separate signing secrets and external DSNs; declaration/card/environment scopes. Managed values belong to Authentication, Service API, Payments or Database. Advanced imports/pending state use Configuration.
SEO	Initial HTML, metadata, panel thresholds and gateway sitemap/robots. Build contracts cover special build placement; Routing and responses covers HTML, CSP and caching.
Site settings	Analytics and badge.
PWA	Manifest ownership.

Configuration and delivery details

Guide	Facts and dependencies
Configuration	Whole imports, interactive cards and stored/applied state. Domain facts belong to Features, Runtime, Hosting, PWA, Secrets, Git canonical/checkpoints, Payments and Shopify; runtime credentials to Service API, logs to Diagnostics, failures to Errors/Project background.
Runtime	Extra Preview connections, proxy behavior and listener recovery. Login/cookies use Authentication; Local loading uses Work Locally, fixed Mobile uses Expo README, advanced state uses Configuration, custom checking uses Diagnostics.
Hosting	Resource sizing, hosting/idle policy and serving observations. Writes use

	Configuration, publication uses Git/checkpoints, logs use Diagnostics, container lifecycle uses the main deployment contract.
Git/checkpoints	Publication, native rollback and Git recovery. Local transport uses Work Locally, repository ownership uses Git canonical, special build failures use Build contracts.
GitHub connection and repository switching	Saving, pushing or connecting the project to the user's GitHub, moving it back to Manus, or completing/canceling that switch; read only when the request involves these operations. A separate backup does not switch the project repository. Config transitions use Configuration; versions/publication use Git/checkpoints; Local transport uses Work Locally.

Exceptional setup and recovery

Guide	Facts and dependencies
Diagnostics	Custom LSP, native settings/reload, failed checking, recovery, published logs and the live version. Configuration and Errors own config/error envelopes.
Build contracts	Nonstandard toolchains, build/startup failures and focused reproduction. Service variables: Service API; policy: Hosting; logs: Diagnostics; process-independent work: Scheduled work; HTTP: Routing and responses; rendering: SEO; advanced writes: Configuration.
Routing and responses	HTML transformation, CSP, advanced caching and route failures. Configuration owns advanced writes and rejected-operation state.
Features	Provisioning and non-Web policies. Use follows Database, Authentication, Payments and Secrets; advanced state uses Configuration, build shape uses Build contracts.

Errors	Concrete error-code lookup pointing directly to its contract/recovery section; do not preload.
Project background	Rare unresolved init/attach, Sandbox or legacy failures; never preload for ordinary work.

Read setup before init. Follow the recorded engine: Three.js uses its starter receipt and Web build/checkpoints; Godot uses its workflow. Both require Game sharing before metadata/OG work or `game/sharing` writes, and in-place, seamless 3D animation loops.

For Godot, read Game Tweak language before creating or editing Tweak parameters, including existing projects and after context loss.